

Esame OS Internals (Cabodi)

Data: 16/01/2023

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, TLB, Cache e EAT)

Contesto: Sia dato un sistema di memoria virtuale con paginazione, nel quale vengono indirizzati i Byte. Il sistema dispone di TLB (Translation Look-aside Buffer), su cui si misura sperimentalmente un "hit ratio" del 90%. La tabella delle pagine ("page-table") viene realizzata con uno schema a due livelli, nel quale un indirizzo logico di 64 bit viene suddiviso (da MSB a LSB) in 3 parti: p_1 , p_2 e d , rispettivamente di 40 bit, 12 bit e 12 bit. Non si utilizzano ulteriori strutture dati (quali tabelle di hash o inverted page table) per velocizzare gli accessi. La memoria virtuale viene gestita con paginazione a richiesta.

Si risponda alle seguenti domande:

-) Supponendo che non sia presente una memoria cache, e che la memoria RAM abbia tempo di accesso di 200 ns, si calcoli il tempo effettivo di accesso (EAT) per il caso proposto (TLB hit ratio = 90%), assumendo che il tempo di accesso alla TLB sia trascurabile.
-) Si vuole ora tener conto del fatto che il sistema dispone di memoria cache, per la quale si sa che la probabilità di (cache) miss è del 10% (un accesso non in cache ogni 10) e che la cache ha tempo di accesso 30 ns. Si ricalcoli EAT, con gli stessi dati per quanto riguarda la TLB e ipotizzando, nel caso di cache miss, un tempo di accesso alla RAM di 230 ns.
-) Si consideri ora la frequenza di page fault p . Si faccia riferimento al caso della domanda B (presenza di cache) e si ipotizzi che un page fault sia servito in 5 ms. Quanto deve valere p affinché si possa garantire un degrado massimo del 25% per EAT (causato sia dalla TLB che dai page fault)? Si tratta di un valore massimo o minimo per p ?

Domanda 2 (File System, Linked List e FAT)

Contesto: Sia dato un file di dimensione 160MB, in un file system di dimensione complessiva 400GB. Si vogliono confrontare le possibili organizzazioni di tale file, su file system con allocazione non contigua, secondo gli schemi:

- "linked list allocation"
- "File Allocation Table (FAT)"

Si supponga che i blocchi su disco (sia per i dati che per i metadati) abbiano dimensione 8KB, e che i puntatori e gli indici abbiano dimensione 32 bit.

Si dica, per ognuna delle 2 soluzioni:

-) Quanti blocchi (di dato e/o di metadato) sono necessari per il file (attenzione: Si richiede il conteggio ESATTO, eventualmente espresso in termini di potenze di 2, ricordando che $1K = 2^{10}$ e $1M = 2^{20}$)? Nel caso della FAT, si dica quale percentuale di FAT è utilizzata per il file.

-) Quante letture in RAM e quanti accessi a disco (per trasferire un blocco in RAM) sono necessari per leggere il byte n. 42070 all'interno del file? Si consideri un accesso diretto al file. Si supponga di non utilizzare buffer cache, e che un puntatore o indice venga letto in RAM con una sola lettura. Si supponga poi che il File Control Block (FCB) sia già in memoria RAM, che ogni accesso a disco legga o scriva un blocco di 8KB, e che la FAT (qualora utilizzata) sia già in RAM.
-

Domanda 3 (Dimensione Pagina e Prestazioni)

Contesto: Si consideri un sistema con paginazione. Si dica, per ognuna delle caratteristiche elencate, se sia preferibile una dimensione di pagina MAGGIORE o MINORE, motivando la risposta.

- Frammentazione (interna)
- Dimensione della Page Table (PT)
- Sovraccarico I/O (I/O overhead)
- Numero di errori di pagina (Page Faults)
- Località
- Dimensioni/copertura TLB

SECTION 2: OS161

Domanda 4 (Creazione Thread in OS161)

Contesto: Si fornisce una rappresentazione (parziale) della funzione OS161 `thread_fork`.

```
int thread_fork(const char *name, struct proc *proc,
               void (*entrypoint)(void *data1, unsigned long data2),
               void *data1, unsigned long data2) {
    struct thread *newthread;
    int result;
    newthread = thread_create(name);

    /* Allocate a stack */
    newthread->t_stack = kmalloc(STACK_SIZE);

    /* Thread subsystem fields */
    newthread->t_cpu = curthread->t_cpu;

    /* Attach the new thread to its process */
    result = proc_addthread(proc, newthread);

    /* Set up the switchframe so entrypoint() gets called */
    switchframe_init(newthread, entrypoint, data1, data2);

    /* Lock the current cpu's run queue and make the new thread runnable */
    thread_make_runnable(newthread, false);

    return 0;
}
```

Basandosi sul codice riportato e su quanto appreso in lezioni e laboratori, si risponda alle seguenti domande:

-) Che cos'è `entrypoint` e a cosa serve nel contesto della creazione di un thread?
-) Lo stack allocato (di dimensione `STACK_SIZE`) è stack kernel oppure user? (Motivare).
-) Che cosa fa la funzione `thread_make_runnable`? Manda in esecuzione il thread? (Se no, motivare; se sì, quando?).
-) E' possibile che l'esecuzione arrivi all'istruzione finale (`return 0;`), oppure, una volta entrati in `thread_make_runnable`, non è possibile che il controllo sia ritornato alla `thread_fork`?

Domanda 5 (Gestione Memoria `dumbvm` e Page Table custom)

Contesto: Si consideri, in OS161, la realizzazione della gestione della memoria virtuale. Si risponda alle seguenti domande:

-) Si supponga di voler realizzare in `dumbvm` la gestione dei frame liberi mediante free-list, invece che con bitmap (come fatto nella soluzione proposta per il lab 2).

- A1)** Si dica se la free list può essere una lista non ordinata oppure se deve essere ordinata. (Motivare).
- A2)** Qualora la lista fosse ordinata, andrebbe ordinata in base a indirizzi fisici oppure logici? (Motivare).
-) Si supponga di voler invece realizzare una PAGE TABLE a livello di singolo processo. Viste le dimensioni ridotte della RAM fisica, si ipotizzi che una casella (entry) della PT occupi solo 2 Byte. Si Ipotizzi di NON voler gestire un heap per il processo: il processo contiene quindi unicamente (come fatto in `dumbvm`) i segmenti di codice, di dati e lo stack. Si sa che:
- La costante `PAGE_SIZE` è definita come 4096.
 - Il segmento 1 (codice) inizia all’indirizzo logico `0x400000` e ha dimensione 10112 Byte.
 - Il segmento 2 (dati) inizia all’indirizzo logico `0x412000` e ha dimensione 8028 Byte.
 - Lo stack ha dimensione 18 pagine, collocate ai massimi indirizzi logici possibili.
- B1)** Si calcoli la dimensione complessiva dell’address space e, al suo interno, il numero di pagine effettivamente usate (valide).
- B2)** Si calcoli quale sarebbe la dimensione di una page table standard.
- B3)** Si calcoli quale sarebbe la dimensione di una page table gerarchica a due livelli, supponendo di utilizzare 11 bit per p_2 : si ricorda che l’indirizzo logico è (p_1, p_2, d) .

ANSWERS

Domanda 1 (Memoria Virtuale, TLB, Cache e EAT)

A) Calcolo EAT senza Cache:

Dati:

- Hit ratio TLB = 90% ($P_{\text{hitTLB}} = 0.9$)
- Miss ratio TLB = 10% ($P_{\text{missTLB}} = 0.1$)
- $T_{\text{RAM}} = 200 \text{ ns}$
- Essendo la Page Table a 2 livelli, la traduzione in caso di miss richiede **2 accessi extra** in RAM.

Il calcolo viene suddiviso logicamente in due fasi: Traduzione e Accesso.

- *Tempo Traduzione* = $P_{\text{hitTLB}} \times 0 + P_{\text{missTLB}} \times (2 \times T_{\text{RAM}}) = 0.1 \times 400 = 40 \text{ ns}$.
- *Tempo Accesso al Dato* = $1 \times T_{\text{RAM}} = 200 \text{ ns}$.

$$EAT = \text{Tempo Traduzione} + \text{Tempo Accesso} = 40 + 200 = \mathbf{240 \text{ ns}}.$$

B) Calcolo EAT con presenza di Cache:

Dati Cache: Hit ratio = 90%, $T_{\text{hit}} = 30 \text{ ns}$, $T_{\text{miss_RAM}} = 230 \text{ ns}$.

Calcoliamo innanzitutto il costo medio per accedere fisicamente alla memoria (T_{MEM}):

$$T_{\text{MEM}} = (0.9 \times 30 \text{ ns}) + (0.1 \times 230 \text{ ns}) = 27 + 23 = \mathbf{50 \text{ ns}}.$$

Assumendo l'architettura standard in cui anche le Page Table beneficiano della Cache, applichiamo nuovamente la suddivisione in fasi sostituendo la RAM pura con T_{MEM} :

- *Tempo Traduzione* = $P_{\text{missTLB}} \times (2 \times T_{\text{MEM}}) = 0.1 \times (2 \times 50) = 0.1 \times 100 = \mathbf{10 \text{ ns}}$.
- *Tempo Accesso al Dato* = $T_{\text{MEM}} = \mathbf{50 \text{ ns}}$.

$$EAT_{\text{conCache}} = \text{Tempo Traduzione} + \text{Tempo Accesso} = 10 + 50 = \mathbf{60 \text{ ns}}.$$

C) Frequenza di Page Fault per un degrado massimo del 25%:

Il "degrado causato sia dalla TLB che dai PF" si valuta rispetto al tempo ideale in cui l'indirizzamento logico non introduce overhead (0 traduzione, 0 PF). Questo tempo ideale è il puro accesso fisico: $T_{\text{MEM}} = 50 \text{ ns}$.

L'EAT massimo tollerato è:

$$EAT_{\text{max}} = 50 + 25\% = 50 \times 1.25 = \mathbf{62,5 \text{ ns}}.$$

Introduciamo il tasso di Page Fault (p) nell'equazione dell'EAT (ricordando che $5 \text{ ms} = 5.000.000 \text{ ns}$):

$$EAT_{\text{Totale}} = EAT_{\text{conCache}} + p \times T_{\text{PF}}$$

Imponiamo il limite massimo:

$$62.5 = 60 + p \times 5.000.000$$

$$2.5 = p \times 5.000.000$$

$$p = \frac{2.5}{5 \times 10^6} = 5 \times 10^{-7}$$

Si tratta di un valore **massimo**. Affinché il degrado rimanga *sotto* il 25%, il tasso di Page Fault deve essere pari o inferiore a questo piccolissimo limite.

Domanda 2 (File System, Linked List e FAT)

Dati di Base:

- Dimensione File = 160 MB = 160×2^{20} Byte = 167.772.160 Byte.
- Dimensione File System = 400 GB = 400×2^{30} Byte.
- Dimensione Blocco = 8 KB = 8×2^{10} Byte = 8192 Byte.
- Dimensione Puntatore = 32 bit = 4 Byte.

A) Conteggio esatto dei blocchi necessari e percentuale FAT:

1. Schema Linked List Allocation: Nello schema a lista concatenata, ogni blocco del disco deve riservare 4 Byte per memorizzare il puntatore al blocco successivo. La dimensione utile del blocco per i dati dell'utente è:

$$\text{Spazio Utile} = 8192 \text{ B} - 4 \text{ B} = 8188 \text{ Byte.}$$

Il numero esatto di blocchi necessari si ottiene dividendo la dimensione del file per lo spazio utile, arrotondando per eccesso:

$$\text{Numero Blocchi} = \left\lceil \frac{167.772.160}{8188} \right\rceil = \lceil 20490,0048 \rceil = \mathbf{20491 \text{ blocchi.}}$$

Essendo i puntatori (metadati) integrati nei blocchi dati, non ci sono ulteriori blocchi di metadati separati da calcolare.

2. Schema File Allocation Table (FAT): Nello schema FAT, i blocchi su disco sono dedicati interamente ai dati. I puntatori risiedono in una tabella separata (la FAT).

$$\text{Blocchi Dati} = \frac{160 \text{ MB}}{8 \text{ KB}} = \frac{160 \times 2^{20}}{8 \times 2^{10}} = 20 \times 2^{10} = \mathbf{20.480 \text{ blocchi.}}$$

(*Calcolo accessorio dei metadati*): Ogni blocco dato richiede un'entry da 4 Byte nella FAT. L'ingombro specifico per questo file all'interno della FAT è $20.480 \times 4 \text{ B} = 81.920 \text{ B} = 80 \text{ KB}$. Corrispondono esattamente a $80 \text{ KB} / 8 \text{ KB} = \mathbf{10 \text{ blocchi di metadati.}}$

Totale spazio richiesto dal file: $20.480 + 10 = \mathbf{20.490 \text{ blocchi.}}$

Calcolo Percentuale FAT utilizzata: Il numero totale di entry nella FAT è pari al numero totale di blocchi indirizzabili nel file system:

$$\text{Entry FAT Totali} = \frac{400 \text{ GB}}{8 \text{ KB}} = \frac{400 \times 2^{30}}{8 \times 2^{10}} = 50 \times 2^{20} = 52.428.800 \text{ entries.}$$

La percentuale di FAT occupata dal file in questione è:

$$\%FAT = \frac{\text{Blocchi Dati File}}{\text{Entry FAT Totali}} = \frac{20.480}{52.428.800} = \frac{1}{2560} = \mathbf{0,039\%}.$$

B) Accesso diretto al byte n. 42070:

1. Schema File Allocation Table (FAT): I blocchi contengono interi 8192 Byte di dati. Il blocco logico che contiene il byte in questione si trova dividendo l'offset:

$$\text{Indice Blocco Logico} = \left\lfloor \frac{42070}{8192} \right\rfloor = 5 \text{ (corrisponde al 6° blocco)}.$$

Essendo la FAT interamente caricata in RAM, il sistema operativo scorre la catena dei puntatori in memoria centrale partendo dal blocco iniziale (fornito dall'FCB).

Sono necessarie **6 letture in RAM** per attraversare la lista collegata nella FAT (per gli indici 0, 1, 2, 3, 4 e 5) e ricavare l'indirizzo fisico corretto. Infine, è necessaria **1 sola lettura su disco** per prelevare il blocco bersaglio.

2. Schema Linked List Allocation: I blocchi contengono solo 8188 Byte di dati utili. Il calcolo dell'indice logico è:

$$\text{Indice Blocco Logico} = \left\lfloor \frac{42070}{8188} \right\rfloor = 5 \text{ (corrisponde al 6° blocco)}.$$

Poiché nello schema Linked List ogni puntatore risiede fisicamente all'interno del blocco precedente su disco, l'accesso casuale (random access) non è supportato. Il sistema operativo è costretto a leggere fisicamente il blocco 0 per scoprire dove si trova il blocco 1, e così via, fino a raggiungere il blocco 5.

Sono necessarie **6 letture su disco** sequenziali. (Zero letture in RAM per scorrimento indici).

Domanda 3 (Dimensione Pagina e Prestazioni)

Analisi delle caratteristiche in base alla dimensione della pagina:

- **Frammentazione (interna) - Preferibile: MINORE.**

Motivazione: La frammentazione interna si verifica fisiologicamente nell'ultima pagina allocata a un processo o segmento. In media, si perde mezza pagina di spazio. Una dimensione di pagina minore riduce matematicamente lo spazio assoluto sprecato.

- **Dimensione della Page Table (PT) - Preferibile: MAGGIORE.**

Motivazione: A parità di spazio di indirizzamento virtuale totale, una dimensione di pagina maggiore riduce il numero complessivo di pagine logiche in cui lo spazio è diviso. Di conseguenza, si riduce il numero di *entry* (righe) necessarie nella Page Table, riducendone l'ingombro in memoria.

- **Sovraccarico I/O (I/O overhead) - Preferibile: MAGGIORE.**

Motivazione: Le operazioni di I/O, specialmente su dischi meccanici, sono dominate da colli di bottiglia fisici (tempo di *seek* e latenza rotazionale). Il tempo di trasferimento effettivo dei byte è marginale. È molto più efficiente (minore overhead) trasferire un grosso blocco di dati in una singola operazione hardware piuttosto che frammentare il trasferimento, il quale costringerebbe il disco a continui riposizionamenti della testina.

- **Numero di errori di pagina (Page Faults) - Preferibile: MAGGIORE.**

Motivazione: Una pagina più grande agisce intrinsecamente come un *read-ahead* hardware. A fronte di un singolo page fault, vengono portati in memoria molti più dati contigui. Sfruttando la località spaziale dei programmi, ciò riduce significativamente la probabilità di incorrere in ulteriori fault per le istruzioni o i dati successivi.

- **Località - Preferibile: MINORE.**

Motivazione: Pagine di dimensioni ridotte permettono di isolare e mappare in RAM con

una granularità più fine i dati effettivamente referenziati (il reale *working set*). Pagine troppo grandi rischiano di includere inutilmente blocchi di codice o dati adiacenti che non verranno mai utilizzati dal processo, spreco di RAM limitata e peggiorando l'efficienza globale del sistema.

- **Dimensioni/copertura TLB - Preferibile: MAGGIORE.**

Motivazione: La copertura hardware della TLB (*TLB Reach*) è determinata dalla formula: Numero di Entry $\times$ Dimensione Pagina. Poiché il numero di entry nella TLB è fisicamente limitato e molto costoso da incrementare, aumentare la dimensione della pagina espande direttamente la quantità di memoria mappata istantaneamente dalla TLB, riducendo i tassi di TLB miss.

Domanda 4 (Creazione Thread in OS161)

- **A) What is the entrypoint?**

The `entrypoint` is a pointer to the function that the new thread should begin executing upon its creation. It serves as the starting address of the thread's execution flow in kernel mode (or user mode, depending on the context of the fork). It is not related to the stack, but rather to the instruction sequence.

- **B) Is the allocated stack a kernel or user stack?**

It is a **kernel stack**. The `thread_fork` function executes within the kernel context, and `kmalloc` allocates memory from the kernel's heap. This stack is used to store local variables, function call frames, and context-save frames (switchframes) for the thread while it runs in kernel mode.

- **C) What does `thread_make_runnable` do? Does it start the thread?**

The function inserts the thread into the CPU's *run queue* (or ready queue), changing its state to `RUNNABLE`. It **does not** start the thread immediately. The thread will only begin executing when the scheduler picks it from the queue and performs a context switch to it.

- **D) Can execution reach the final `return 0;` statement?**

YES. The execution of the calling thread (the one invoking `thread_fork`) continues normally after calling `thread_make_runnable`. It reaches the `return 0;` statement and completes its own execution flow. The new thread created is simply added to the scheduler's queue, and it will eventually start its own independent execution flow starting from the `entrypoint` function, unrelated to the code path of the creator thread.

Domanda 5 (Gestione Memoria `dumbvm` e Page Table custom)

A) Free-list in `dumbvm`:

- **A1) Lista non ordinata o ordinata? Ordinata.**

Motivazione: L'allocatore fisico `getppages(n)` di OS161 `dumbvm` necessita di allocare n frame fisici in RAM che siano **strettamente contigui**. Una lista disordinata richiederebbe una scansione estremamente onerosa per individuare blocchi di frame contigui sufficientemente grandi. L'ordinamento garantisce efficienza sia per la ricerca di porzioni contigue (First-Fit, Best-Fit) sia, soprattutto, per la *coalescenza* (l'unione di frame adiacenti liberati) in tempo $O(N)$ o inferiore.

- **A2) Ordinata per indirizzi fisici o logici? Indirizzi Fisici.**

Motivazione: La free-list gestisce la memoria RAM fisica hardware, la quale esiste indipendentemente dagli spazi di indirizzamento virtuali creati dal kernel per i singoli processi. Inoltre, l'ordinamento per indirizzo fisico è l'unica condizione che rende possibile la coalescenza dei frame adiacenti.

B) Page Table a singolo processo (Entry = 2 Byte, Pagina = 4 KB):

Dati pre-calcolati per i segmenti:

- Pagine Codice = $\lceil 10112/4096 \rceil = 3$ pagine.
- Pagine Dati = $\lceil 8028/4096 \rceil = 2$ pagine.
- Pagine Stack = **18** pagine.

1. B1) Dimensione Address Space e Pagine Valide:

In un sistema MIPS32 (come OS161), lo spazio di indirizzamento utente (kuseg) parte da 0x00000000 e termina all'indirizzo 0x7FFFFFFF (poiché a 0x80000000 inizia il kseg0 del kernel).

Dimensione complessiva Address Space = **2 GB (circa 2^{31} Byte)**.

Le pagine effettivamente usate all'interno di questo immenso spazio sono solo quelle allocate ai segmenti:

Pagine Valide = 3 (codice) + 2 (dati) + 18 (stack) = **23** pagine.

2. B2) Dimensione di una Page Table Standard (Single Level):

Una Page Table Standard è un array indicizzato (Flat Array) che deve necessariamente contenere un'entry per **ogni singola pagina logica** dell'intero Address Space, a prescindere dal fatto che sia usata o meno.

$$\text{Numero Entry Totali} = \frac{2 \text{ GB}}{4 \text{ KB}} = \frac{2^{31}}{2^{12}} = 2^{19} = \mathbf{524.288 \text{ entries.}}$$

$$\text{Dimensione PT Standard} = 524.288 \times 2 \text{ Byte} = 1.048.576 \text{ Byte} = \mathbf{1 \text{ MB.}}$$

3. B3) Dimensione di una Page Table Gerarchica (2 Livelli):

Scomposizione dei 32 bit logici:

- Offset (d): La pagina è 4 KB (2^{12} B), quindi $d = 12$ bit.
- Indice di secondo livello (p_2): Fornito dal testo, $p_2 = 11$ bit.
- Indice di primo livello (p_1): Ricavato per sottrazione, $p_1 = 32 - 12 - 11 = 9$ bit.

Calcolo della Directory di primo livello (P1): C'è sempre esattamente **una** directory P1.

$$\text{Entry in P1} = 2^{p_1} = 2^9 = 512 \text{ entries.}$$

$$\text{Dimensione P1} = 512 \times 2 \text{ Byte} = \mathbf{1024 \text{ Byte.}}$$

Calcolo delle Tabelle di secondo livello (P2) necessarie:

Una singola tabella P2 contiene $2^{p_2} = 2^{11} = 2048$ entries. Copre un intervallo di memoria di $2048 \times 4 \text{ KB} = 8 \text{ MB}$.

- Il segmento Codice (inizia a 4 MB) e il segmento Dati (immediatamente successivo) ricadono entrambi all'interno del primo intervallo di 8 MB coperto dalla primissima tabella P2. $\Rightarrow$ **1 Tabella P2 allocata.**
- Lo stack si trova ai massimi indirizzi logici possibili, ricadendo quindi nell'ultima tabella P2 possibile. $\Rightarrow$ **1 Tabella P2 allocata.**

Servono in totale solo **2 tabelle P2** (su 512 possibili).

$$\text{Dimensione Tabelle P2} = 2 \times (2048 \times 2 \text{ Byte}) = \mathbf{8192 \text{ Byte.}}$$

Dimensione Totale Page Table Gerarchica:

$$\text{Dimensione Totale} = \text{Dimensione P1} + \text{Dimensione P2} = 1024 \text{ B} + 8192 \text{ B} = \mathbf{9216 \text{ Byte.}}$$